

Ledger

Distributed Payments & Settlement Engine

Client	Business Bot Technologies (internal R&D initiative)	Document type	Software Requirements & Build Specification
Prepared for	Contracted Developer	Status	Approved for build
Engagement type	Fixed-scope build, phased delivery	Version	1.0

1. What Ledger Is

Ledger is a backend payments and settlement engine — the same category of system as Stripe, Paystack, or Plaid, scoped down to a buildable but production-representative size. It sits behind a business application (such as a merchant dashboard or a platform like Business Bot) and is responsible for one job only: recording money movement correctly, once, and being able to prove it.

It is not an accounting app with a balance column that goes up and down. Ledger uses **double-entry bookkeeping**: every transaction is recorded as a matched pair of entries (a debit and a credit) against immutable records. A balance is never stored directly — it is always *calculated* by summing entries. This is the same principle every real bank, card network, and payment processor is built on, because it makes the ledger self-auditing: if the sums of debits and credits ever stop matching, you know immediately that something is broken.

Around that core, Ledger behaves like a real payments platform: it accepts transaction requests from client applications, processes them safely even when networks fail or requests are retried, notifies merchants of events in real time, delivers webhooks to external systems the way Ledger's own upstream providers (mobile money aggregators, card processors) would, and flags suspicious activity automatically.

The purpose of this build is explicit: it is a deliberately harder engineering problem than a typical CRUD SaaS product, chosen to force correctness-under-failure thinking — idempotency, partial-failure handling, eventual consistency, and auditability — rather than another feature-complete but architecturally simple application.

2. Business Objective

- Provide a single, reliable source of truth for money movement that any client application (merchant dashboards, mobile apps, internal tools) can read from and write to without ever double-processing a payment.
- Guarantee that no transaction can be lost, duplicated, or left in an inconsistent state, even under network failure, service crashes, or duplicate client requests.
- Give merchants real-time visibility into their transactions and balances, not delayed batch reporting.
- Notify external systems (merchant webhooks) of transaction events reliably, with retry and proof of delivery.
- Detect anomalous or likely-fraudulent transaction patterns automatically and flag them for review.
- Produce a fully traceable audit trail: for any transaction, it must be possible to reconstruct exactly what happened, in what order, and why.

3. Scope of Work

In scope

- Core ledger service: accounts, transactions, double-entry postings, balance computation

- Idempotent transaction API with duplicate-request protection
- Asynchronous event pipeline for transaction-created, transaction-settled, and transaction-flagged events
- Real-time merchant dashboard feed via WebSockets
- Outbound webhook delivery system with retries and signature verification
- Rule-based and model-assisted fraud/anomaly detection layer
- Authentication, authorization, and per-merchant multi-tenancy
- Rate limiting on all write and payment-adjacent endpoints
- Structured logging and distributed tracing across the full request/event lifecycle
- Admin/reporting endpoints for reconciliation and audit

Out of scope (explicitly not required for this build)

- Real integration with an actual mobile money or card network — external providers are simulated/mock
- A production frontend UI beyond a minimal dashboard needed to demonstrate the real-time feed
- Multi-currency FX conversion logic (single-currency ledger is sufficient; schema should not preclude future multi-currency support)
- PCI-DSS-level compliance tooling — security practices should follow good hygiene, not a compliance audit

4. Functional Requirements

4.1 Accounts & Ledger Core

- System shall support creation of **Accounts** (e.g. merchant wallet, platform revenue account, escrow/holding account), each with a currency and account type.
- System shall record every movement of money as a **Transaction** composed of two or more **LedgerEntries** (debits and credits) that sum to zero across the transaction.
- Transactions and ledger entries shall be immutable once created — corrections are made via new reversing/compensating entries, never by editing or deleting existing records.
- Account balance shall always be a computed value (sum of entries), never a stored/mutated field.
- System shall support at minimum: deposit, withdrawal, transfer-between-accounts, and reversal transaction types.

4.2 Idempotency & Request Safety

- Every transaction-creating endpoint shall require a client-supplied **Idempotency-Key**.
- A repeated request with the same idempotency key shall return the original response and shall not create a duplicate transaction, regardless of how many times it is retried.
- Idempotency records shall expire after a defined retention window (e.g. 24–72 hours) and shall store enough of the original request/response to safely replay it.

4.3 Asynchronous Event Processing

- Every state-changing operation on the ledger shall write an event to an **outbox table** within the same database transaction as the ledger write itself, so the event can never be lost or desynced from the data it describes.
- A background worker shall poll/consume the outbox and publish events onto a stream (e.g. Redis Streams) for downstream consumers.
- Consumers (fraud check, notification dispatch, webhook delivery) shall be idempotent — safe to process the same event more than once without side effects.
- System shall support at minimum these event types: `transaction.created`, `transaction.settled`, `transaction.reversed`, `transaction.flagged`.

4.4 Real-Time Notifications

- Merchant-facing dashboard shall receive live updates over WebSocket when a transaction affecting their account is created, settled, or flagged — no polling.
- Connection shall be authenticated and scoped to the merchant's own account/tenant only.
- System shall handle reconnect gracefully and shall not replay a backlog of every historical event on reconnect — only a bounded recent window plus current state.

4.5 Outbound Webhooks

- Merchants shall be able to register a webhook URL to receive transaction event notifications.
- Webhook payloads shall be signed (HMAC) so receivers can verify authenticity.
- Failed deliveries shall be retried with exponential backoff up to a defined maximum attempt count.
- System shall keep a delivery log per webhook attempt (status, response code, timestamp, attempt number) for merchant-facing debugging.

4.6 Fraud & Anomaly Detection

- System shall evaluate incoming transactions against a rule set (e.g. velocity limits, duplicate-amount clustering, unusual-hour activity, first-time-large-transaction) and shall flag matches for review rather than blocking automatically, unless a rule is explicitly marked hard-block.
- Flagged transactions shall be visible in a review queue with the reason(s) they were flagged.
- System shall support an assisted-review layer (model- or LLM-based) that produces a plain-language explanation of why a transaction pattern looks anomalous, to support human reviewers rather than replace them.

4.7 Authentication, Authorization & Multi-Tenancy

- System shall support merchant-level accounts with JWT-based authentication and refresh token rotation.
- System shall enforce that a merchant can only ever read or act on their own accounts and transactions — cross-tenant access must be impossible at the query layer, not just hidden in the UI.
- System shall support at least two roles per merchant tenant: owner (full access) and viewer (read-only).

4.8 Audit & Reconciliation

- System shall expose an endpoint/report that verifies ledger integrity: total debits equal total credits across the whole system at any point in time.
- Every transaction shall be traceable end-to-end: its originating request, its ledger entries, every event it produced, and every webhook delivery attempt tied to it.
- System shall retain a complete, append-only audit log of all financial state changes.

5. Non-Functional Requirements

Category	Requirement
Consistency	No transaction may ever result in unbalanced debits/credits. Correctness under concurrent requests takes priority over raw throughput.
Reliability	A crash of any single component (API, worker, event consumer) must not lose data or leave a transaction half-processed with no recovery path.
Observability	Every request must be traceable across services via a correlation/trace ID. Structured logs required on every service. Distributed tracing (e.g. OpenTelemetry) across the ledger → event bus → webhook delivery path.
Security	All write endpoints authenticated. Rate limiting on all payment-adjacent and auth endpoints. Webhook payloads signed. Secrets never logged.
Performance	Ledger write path should remain correct and responsive under concurrent writes to the same account (no lost updates/race conditions on balance-affecting operations).
Auditability	All financial records immutable. All corrections via compensating entries. Full history reconstructable.

6. Technical Requirements

Layer	Requirement
API framework	FastAPI, fully async
Database	PostgreSQL, accessed via async SQLAlchemy 2.0. Alembic as the sole migration path.
Event bus	Redis Streams, using consumer groups for at-least-once delivery to multiple independent consumers.
Background processing	Celery (or equivalent) for outbox polling, webhook delivery workers, and scheduled reconciliation jobs.
Real-time layer	WebSockets, authenticated per connection, scoped per tenant.
Caching / rate limiting	Redis-backed rate limiting (token bucket, atomic via Lua script). Redis caching for read-heavy, rarely-changing lookups only — never for balances.
Tracing / logging	OpenTelemetry for distributed tracing; structured (JSON) logging with correlation IDs on every service.
Fraud/AI layer	Rule engine for deterministic checks; LLM-assisted explanation layer for flagged-transaction review (retrieval over historical flagged cases is acceptable here).
Containerization	Docker for all services; docker-compose for local multi-service orchestration.

6.1 Core Data Model (minimum)

- **Account** — id, tenant_id, type, currency, created_at
- **Transaction** — id, type, status, idempotency_key (unique), created_at, metadata
- **LedgerEntry** — id, transaction_id, account_id, direction (debit/credit), amount, created_at
- **OutboxEvent** — id, aggregate_id, event_type, payload, published (bool), created_at
- **WebhookEndpoint** — id, tenant_id, url, secret, active
- **WebhookDelivery** — id, webhook_endpoint_id, event_id, status, attempt_count, last_attempt_at
- **FraudFlag** — id, transaction_id, reason, severity, reviewed (bool)

7. Architecture Overview

Client request arrives at the FastAPI ledger service with an idempotency key. The service checks for an existing result under that key; if none exists, it opens a single database transaction that (a) writes the balanced ledger entries and (b) writes a corresponding row to the outbox table, then commits both atomically. A separate background worker polls the outbox, publishes unpublished events to a Redis Stream, and marks them published. Independent consumer groups on that stream handle fraud scoring, webhook delivery, and WebSocket push to connected dashboards — each consumer is independently scalable and independently failable without affecting the ledger's correctness.

This separation is the whole point: the ledger write path only ever has to guarantee one thing (balanced, idempotent, atomic writes), and everything else — notifications, fraud checks, webhooks — is downstream, asynchronous, and allowed to be eventually consistent.

8. Development Phases & Milestones

Phase 1 — Ledger Core

Deliverables: Account and transaction models, double-entry posting logic, idempotency key handling, balance computation, integrity-check endpoint (debits = credits).

Definition of done: *A transaction cannot be double-processed even when the identical request is fired concurrently multiple times.*

Phase 2 — Outbox & Event Bus

Deliverables: Outbox table wired into the same transaction as ledger writes; background worker publishing to Redis Streams; at least one consumer group proven end-to-end.

Definition of done: *Killing the ledger service mid-write never produces an orphaned event or a lost one.*

Phase 3 — Async Workers & Webhooks

Deliverables: Celery workers for outbox polling and webhook delivery; HMAC signing; retry with exponential backoff; delivery log.

Definition of done: *A merchant's webhook endpoint that is temporarily down still eventually receives the event, with proof of every attempt.*

Phase 4 — Real-Time Dashboard

Deliverables: Authenticated, tenant-scoped WebSocket channel; minimal dashboard showing live balance and transaction feed.

Definition of done: *A transaction created via the API appears on the dashboard without a page refresh or poll.*

Phase 5 — Fraud & Anomaly Layer

Deliverables: Deterministic rule engine (velocity, duplicate-amount, off-hours checks); review queue; LLM-assisted explanation for flagged items.

Definition of done: *A crafted suspicious transaction pattern is correctly flagged with a human-readable reason.*

Phase 6 — Observability & Hardening

Deliverables: Distributed tracing across the full path, structured logging, rate limiting on all write/auth endpoints, load test of concurrent writes to a single account.

Definition of done: *One trace ID can be followed from the original API call through every event and webhook it produced.*

9. Acceptance Criteria (System-Level)

- Firing the same transaction request 50 times concurrently with the same idempotency key results in exactly one transaction and one set of ledger entries.
- At any point in time, the sum of all debit entries equals the sum of all credit entries system-wide.
- Hard-killing the API process mid-request never leaves a ledger entry without a matching outbox event, or vice versa.
- A merchant dashboard reflects a new transaction in real time without refresh or polling.
- A webhook endpoint that returns errors for the first three attempts still eventually receives the event, and the delivery log shows all attempts.
- A deliberately crafted anomalous transaction pattern is flagged with a stated reason, and appears in the review queue.
- No API call can return or modify data belonging to a tenant other than the authenticated caller's own.
- A single trace ID connects the original HTTP request, its DB writes, its published event, and any webhook deliveries it triggered.

10. Glossary

Term	Meaning
Double-entry bookkeeping	Accounting method where every transaction is recorded as matched debit and credit entries that always sum to zero, making the ledger self-verifying.
Idempotency key	A client-supplied unique identifier for a request, used to safely allow retries without duplicate processing.
Outbox pattern	Writing an event to a database table in the same transaction as the business data change, so the event can be published reliably afterward without risking data/event desync.
Eventual consistency	A system property where downstream state (e.g. a webhook, a dashboard) may lag briefly behind the source of truth but is guaranteed to catch up.
Consumer group	A named group of consumers reading from a stream such that each message is delivered to the group at least once, enabling independent, scalable processing.
Saga / compensating transaction	A pattern for correcting or undoing a multi-step operation by issuing new offsetting actions, rather than editing or deleting the original records.

End of specification. Phases are intended to be built and demoed sequentially; each phase's definition of done should be verifiable before moving to the next.